Name: Benedicto, Ruel Jr A.	Date Performed: 12/16/2025
Course/Section: CPE212-CPE31S1	Date Submitted: 12/16/2025
Instructor: Engr. Robin Valenzuela	Semester and SY: 2nd Sem 2025-2026

Activity 2: SSH Key-Based Authentication and Setting up Git

1. Objectives:

1.1 Configure remote and local machine to connect via SSH using a KEY instead of using a password
1.2 Create a public key and private key
1.3 Verify connectivity
1.4 Setup Git Repository using local and remote repositories
1.5 Configure and Run ad hoc commands from local machine to remote servers

Part 1: Discussion

It is assumed that you are already done with the last Activity (**Activity 1: Configure Network using Virtual Machines).** *Provide screenshots for each task*.

It is also assumed that you have VMs running that you can SSH but require a password. Our goal is to remotely login through SSH using a key without using a password. In this activity, we create a public and a private key. The private key resides in the local machine while the public key will be pushed to remote machines. Thus, instead of using a password, the local machine can connect automatically using SSH through an authorized key.

What Is ssh-keygen?

Ssh-keygen is a tool for creating new authentication key pairs for SSH. Such key pairs are used for automating logins, single sign-on, and for authenticating hosts.

SSH Keys and Public Key Authentication

The SSH protocol uses public key cryptography for authenticating hosts and users. The authentication keys, called SSH keys, are created using the keygen program.

SSH introduced public key authentication as a more secure alternative to the older .rhosts authentication. It improved security by avoiding the need to have passwords stored in files and eliminated the possibility of a compromised server stealing the user's password.

However, SSH keys are authentication credentials just like passwords. Thus, they must be managed somewhat analogously to usernames and passwords. They

should have a proper termination process so that keys are removed when no longer needed.

Task 1: Create an SSH Key Pair for User Authentication

1. The simplest way to generate a key pair is to run *ssh-keygen* without arguments. In this case, it will prompt for the file in which to store keys. First, the tool asked where to save the file. SSH keys for user authentication are usually stored in the users .ssh directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case *id_rsa* when using the default RSA algorithm. It could also be, for example, *id_dsa* or *id_ecdsa*.
2. Issue the command *ssh-keygen -t rsa -b 4096.* The algorithm is selected using the -t option and key size using the -b option.

```
benedicto@Workstation:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/benedicto/.ssh/id_rsa):
/home/benedicto/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/benedicto/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/benedicto/.ssh/id_rsa
Your public key has been saved in /home/benedicto/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:w52nLkrWwEM3N1mbu2cmpglM0DIRxZkK9VNHceYdJtM benedicto@Workstation
The key's randomart image is:
+---[RSA 4096]----+
|      .++.o..=++o |
|     .  ++. + *+E.|
|      .=.* + o . .|
|     o.* = o .    |
|      + S o o     |
|       * . o .    |
|      o + . + +   |
|     o  .o + =    |
|      .. .+       |
+----[SHA256]-----+
benedicto@Workstation:~$
```

3. When asked for a passphrase, just press enter. The passphrase is used for encrypting the key, so that it cannot be used even if someone obtains the private key file. The passphrase should be cryptographically strong.
4. Verify that you have created the key by issuing the command *ls -la .ssh.* The command should show the .ssh directory containing a pair of keys. For example, id_rsa.pub and id_rsa.

```
benedicto@Workstation:~$ ls -la .ssh
total 36
drwx------  2 benedicto benedicto 4096 Dec 13 16:32 .
drwxr-x--- 22 benedicto benedicto 4096 Dec 15 11:09 ..
-rw-------  1 benedicto benedicto  757 Dec 13 13:32 authorized_keys
-rw-------  1 benedicto benedicto  411 Jan  8 10:31 id_ed25519
-rw-r--r--  1 benedicto benedicto  103 Jan  8 10:31 id_ed25519.pub
-rw-------  1 benedicto benedicto 3389 Jan  8 10:33 id_rsa
-rw-r--r--  1 benedicto benedicto  747 Jan  8 10:33 id_rsa.pub
-rw-------  1 benedicto benedicto 2240 Dec 13 16:32 known_hosts
-rw-------  1 benedicto benedicto 1404 Dec 13 16:32 known_hosts.old
benedicto@Workstation:~$ S
```

Task 2: Copying the Public Key to the remote servers

1. To use public key authentication, the public key must be copied to a server and installed in an *authorized_keys* file. This can be conveniently done using the *ssh-copy-id* tool.

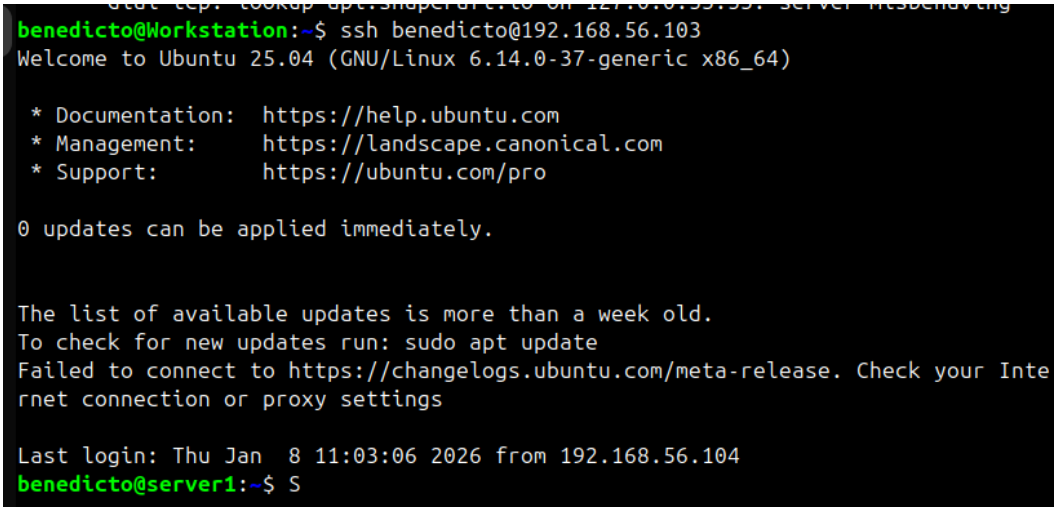
```
benedicto@Workstation:~$ ssh-copy-id
Usage: /usr/bin/ssh-copy-id [-h|-?|-f|-n|-s|-x] [-i [identity_file]] [-t target_
path] [-F ssh_config] [[-o ssh_option] ...] [-p port] [user@]hostname
        -f: force mode -- copy keys without trying to check if they are already
installed
        -n: dry run    -- no keys are actually copied
        -s: use sftp   -- use sftp instead of executing remote-commands. Can be
useful if the remote only allows sftp
        -x: debug      -- enables -x in this shell, for debugging
        -h|-?: print this help
```

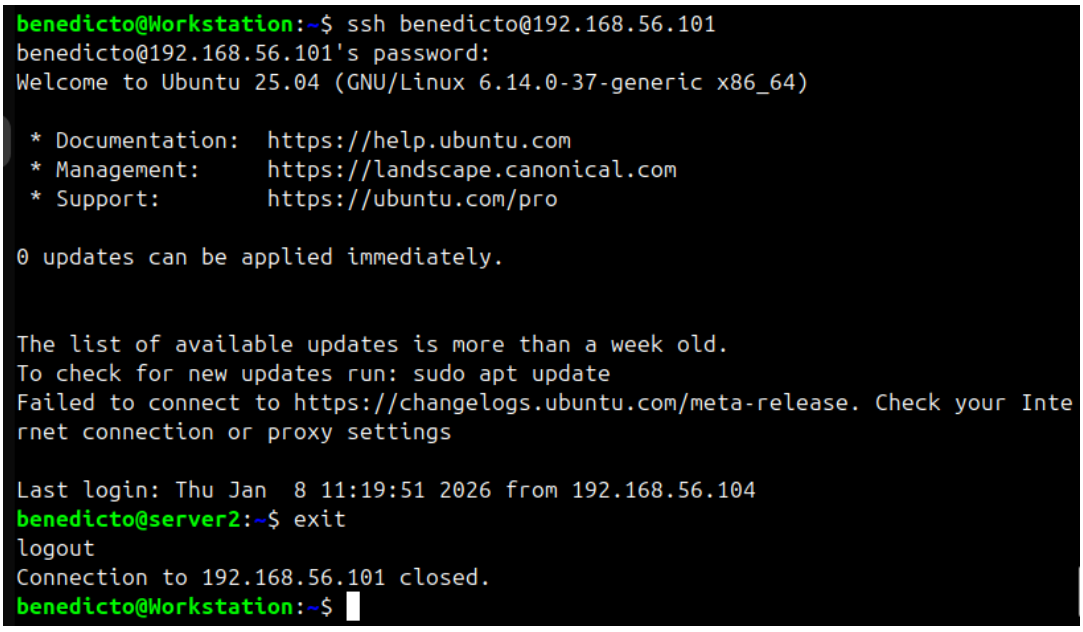
2. Issue the command similar to this: *ssh-copy-id -i ~/.ssh/id_rsa user@host*

```
benedicto@Workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa benedicto@192.168.56.102
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/benedicto/.
ssh/id_rsa.pub"
The authenticity of host '192.168.56.102 (192.168.56.102)' can't be established.
ED25519 key fingerprint is SHA256:st9PBeYo7f4e5ifZBWa/EVV4Am1EBqqrGztysCs7UwY.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:1: [hashed name]
    ~/.ssh/known_hosts:4: [hashed name]
    ~/.ssh/known_hosts:5: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])?
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
    ~/.ssh/known_hosts:5: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt
ed now it is to install the new keys
benedicto@192.168.56.102's password:
Permission denied, please try again.
benedicto@192.168.56.102's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /home/benedicto/.ssh/id_rsa 'ben
edicto@192.168.56.102'"
and check to make sure that only the key(s) you wanted were added.
```

3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.
    4. On the local machine, verify that you can SSH with Server 1 and Server 2. What did you notice? Did the connection ask for a password? If not, why?

- **In server 1 It doesn't ask for password but In server 2 It asks for password. Both connects successfully**





Reflections:
Answer the following:
1. How will you describe the ssh-program? What does it do?
   - ssh is a secure shell program and it provides secure communications between 2 devices. It operates in a client-server model where the client is trying to connect to the server.
2. How do you know that you already installed the public key to the remote servers?

```
benedicto@Workstation:~$ ls -la .ssh
total 36
drwx------  2 benedicto benedicto 4096 Dec 13 16:32 .
drwxr-x--- 22 benedicto benedicto 4096 Dec 15 11:09 ..
-rw-------  1 benedicto benedicto  757 Dec 13 13:32 authorized_keys
-rw-------  1 benedicto benedicto  411 Jan  8 10:31 id_ed25519
-rw-r--r--  1 benedicto benedicto  103 Jan  8 10:31 id_ed25519.pub
-rw-------  1 benedicto benedicto 3389 Jan  8 10:33 id_rsa
-rw-r--r--  1 benedicto benedicto  747 Jan  8 10:33 id_rsa.pub
-rw-------  1 benedicto benedicto 2240 Dec 13 16:32 known_hosts
-rw-------  1 benedicto benedicto 1404 Dec 13 16:32 known_hosts.old
benedicto@Workstation:~$ S
```

ls -ls .ssh shows that the private key and public key exists and this command also shows the exact date and time the key is created.

Part 2: Discussion

Provide screenshots for each task.

It is assumed that you are done with the last activity (**Activity 2: SSH Key-Based Authentication**).

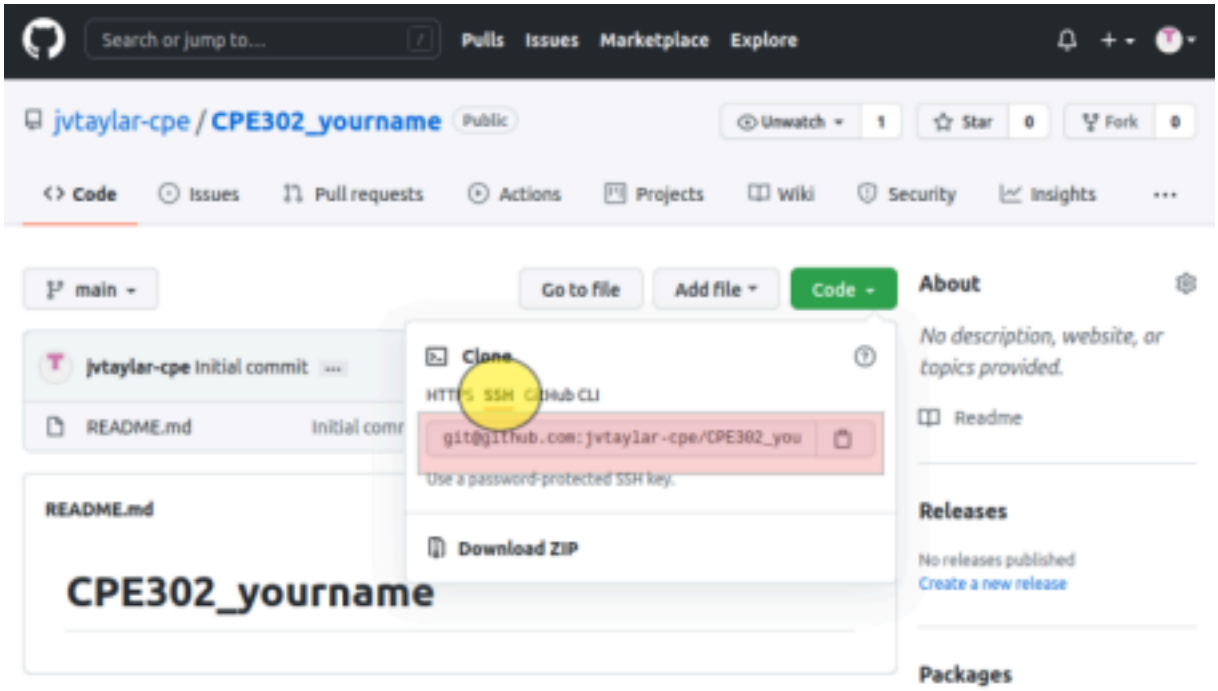
Set up Git
At the heart of GitHub is an open-source version control system (VCS) called Git. Git is responsible for everything GitHub-related that happens locally on your computer. To use Git on the command line, you'll need to download, install, and configure Git on your computer. You can also install GitHub CLI to use GitHub from the command line.

If you don't need to work with files locally, GitHub lets you complete many Git-related actions directly in the browser, including:

- Creating a repository
- Forking a repository
- Managing files
- Being social

Task 3: Set up the Git Repository

1. On the local machine, verify the version of your git using the command *which git*. If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: *sudo apt install git*
2. After the installation, issue the command *which git* again. The directory of git is usually installed in this location: *user/bin/git*.
3. The version of git installed in your device is the latest. Try issuing the command *git --version* to know the version installed.
4. Using the browser in the local machine, go to www.github.com. 5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub account.
    a. Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.
    b. Create a new SSH key on GitHub. Go your profile's setting and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create a new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.
    c. On the local machine's terminal, issue the command cat .ssh/id_rsa.pub and copy the public key. Paste it on the GitHub key and press Add SSH key.
    d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.



e. Issue the command git clone followed by the copied link. For example, *git clone* *git@github.com:jvtaylar-cpe/CPE232_yourname.git*. When prompted to continue connecting, type yes and press enter.

```
benedicto@Workstation:~$ git clone git@github.com:benedictocyan/cpe232_ruel.git
Cloning into 'cpe232_ruel'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
benedicto@Workstation:~$
```

f. To verify that you have cloned the GitHub repository, issue the command *ls*. Observe that you have the CPE232_yourname in the list of your directories. Use CD command to go to that directory and LS command to see the file README.md.

```
benedicto@Workstation:~$ ls
'2025-11-19 20-03-42.mkv'   Downloads          scriptapt.sh
 basicscript.sh             final_project.sh   scriptbackup.sh
 BenedictoJr.s              getrichquick.sh    scriptcheck.sh
 BenedictoJr.sh             gmail.txt          scriptcopy.sh
 CPE211                     Jr..txt.bz2        scriptmenu.sh
 CPE212_BENEDICTOJR         last_name          scriptoddeven.sh
 cpe232_ruel                Music              scriptpass.sh
 cyan                       mybackups          snap
 cyan.sh                    myfiles            Templates
 Desktop                    Pictures           try.sh
 display.sh                 practice1.sh       try.sh.save
 Documents                  Public             Videos
benedicto@Workstation:~$
```

g. Use the following commands to personalize your git.

- *git config --global user.name "Your Name"*
- *git config --global user.email yourname@email.com*
- Verify that you have personalized the config file using the command *cat ~/.gitconfig*

```
benedicto@Workstation:~$ git config --global user.email qrabenedictojr@tip.edu.p
h
benedicto@Workstation:~$ car ~/.gitconfig
Command 'car' not found, but there are 15 similar ones.
benedicto@Workstation:~$ cat ~/.gitconfig
[user]
        name = benedicto
        email = qrabenedictojr@tip.edu.ph
benedicto@Workstation:~$
```

h. Edit the README.md file using nano command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.

i. Use the *git status* command to display the state of the working directory and the staging area. This command shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?

```
benedicto@Workstation:~/cpe232_ruel$ nano README.md
benedicto@Workstation:~/cpe232_ruel$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
benedicto@Workstation:~/cpe232_ruel$
```

j. Use the command *git add README.md* to add the file into the staging area.

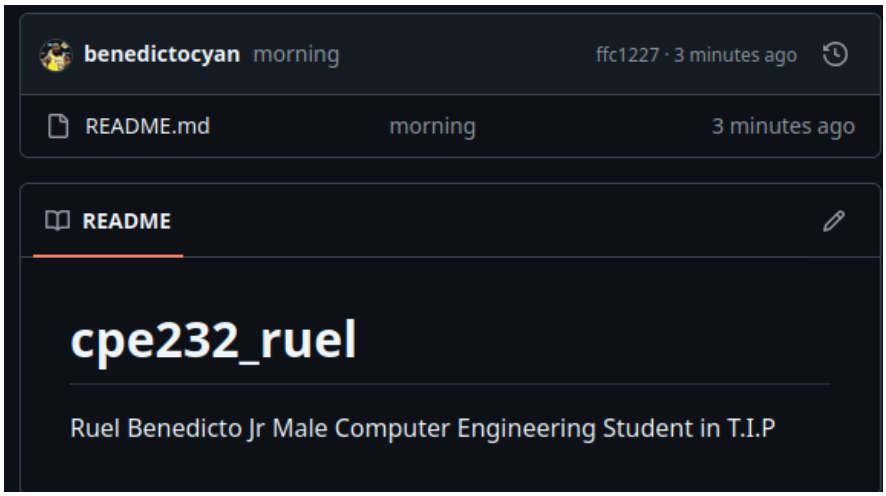
k. Use the *git commit -m "your message"* to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next commit.

```
benedicto@Workstation:~/cpe232_ruel$ nano README.md
benedicto@Workstation:~/cpe232_ruel$ git add README.md
benedicto@Workstation:~/cpe232_ruel$ git commit -m morning morning
error: pathspec 'morning' did not match any file(s) known to git
benedicto@Workstation:~/cpe232_ruel$ git commit -m morning
[main ffc1227] morning
 1 file changed, 4 insertions(+), 1 deletion(-)
benedicto@Workstation:~/cpe232_ruel$
```

l. Use the command *git push <remote><branch>* to upload the local repository content to GitHub repository. Pushing means to transfer commits from the local repository to the remote repository. As an example, you may issue *git push origin main*.

```
benedicto@Workstation:~/cpe232_ruel$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 313 bytes | 313.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:benedictocyan/cpe232_ruel.git
   837f46c..ffc1227  main -> main
```

m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice the how long was the last commit. It should be some minutes ago and the message you typed on the git commit command should be there. Also, the README.md file should have been edited according to the text you wrote.



Reflections:

Answer the following:

3. What sort of things have we so far done to the remote servers using ansible commands?

- Using Ansible commands, we have remotely run tasks such as installing packages, configuring files, managing services, and updating system settings on the servers.

4. How important is the inventory file?

- The inventory file is important because it lists the servers ansible should manage. It serves at the primary address book that defines which systems the tool should manage and how to connect them.

Conclusions/Learnings: